智汇研途 • 作品介绍文档

2026 第九届传智杯全国大学生数智创新与 AI 应用大赛 • AI 大模型创新应用挑战赛（个人赛）

一、项目详情及使用方式

1.1 项目背景与定位

在学术科研场景中，科研人员每天面对海量论文（如 arXiv 日均更新数百篇），「查文献 → 读文献 → 写综述」是一个极其耗时且重复的过程。与此同时，通用大模型虽然能快速生成文字，却存在两个致命问题：一是「幻觉」——会一本正经地编造不存在的文献和结论；二是「无溯源」——回答缺少出处，无法核验真伪，这在要求严谨的学术场景中是不可接受的。

「智汇研途」正是为此设计：它把大模型的生成能力与「检索—总结—批判」的多智能体工作流、以及基于向量库的 RAG 检索增强结合起来，让 AI 的每一次回答都尽量「先查证、再作答、可追溯」。

1.2 系统组成与架构

系统采用前后端分离架构，自上而下分为四层：

- 表现层（前端）：原生 HTML + TailwindCSS + Chart.js，负责三栏交互界面、消息渲染与智能体工作流可视化。
- 服务层（后端）：FastAPI 提供 RESTful 接口，封装 DeepSeek API 调用与 RAG 检索。
- 检索层（向量库）：ChromaDB 持久化存储文献片段，提供语义相似度检索。
- 模型层：DeepSeek Chat API 提供大模型推理能力。

在服务层内部，平台将一次科研问答编排为三个智能体的接力协作：检索员（Retriever）负责找证据，总结员（Summarizer）负责归纳整理，批判员（Critic）负责逻辑与引用校验，形成「检索→总结→批判」的质量闭环。前端通过 HTTP 调用后端接口，后端负责鉴权、重试与检索增强，前后端解耦、职责清晰。

1.3 使用方式

环境要求：Python 3.9+、可联网浏览器。具体步骤：

1. 安装后端依赖：pip install -r requirements.txt

2. 配置 API Key：复制 backend/.env.example 为 backend/.env，填入你的 DeepSeek API Key
3. 启动后端服务：cd backend，然后执行 uvicorn main:app --reload，服务默认运行在 http://127.0.0.1:8000
4. 打开前端：用浏览器打开 frontend/index.html，输入研究问题即可开始使用
5. 上传文献：点击左侧「上传文献 PDF」导入文献，即可进行溯源问答与大纲生成

使用说明：前端会优先调用后端真实接口；若后端未启动，前端会自动回退到模拟回复，保证演示不中断。后端接口文档可在 http://127.0.0.1:8000/docs 查看。

1.4 开源地址

项目已开源至 GitHub（Public）：https://github.com/J0727/-ai-

二、项目的亮点

2.1 十大核心亮点

- 多智能体协作，三层质检：检索员、总结员、批判员分工明确，上一环节输出作为下一环节输入，解决「单模型一次性生成容易出错」的问题。
- RAG 检索增强抗幻觉：先检索真实文献片段、再交给大模型生成，从源头抑制「编造文献」的幻觉问题。
- 智能体工作流可视化：右侧面板用实时动画展示三个智能体的「等待 / 进行中 / 已完成」状态，把看不见的推理过程变成看得见的协作流程，解决「AI 黑盒」的信任问题。
- 引用来源可视化：Chart.js 甜甜圈图直观展示 arXiv、PubMed、IEEE Xplore 等来源占比，让结论具备可追溯性。
- 真实大模型接入：后端封装 DeepSeek Chat API，含 30 秒超时控制与指数退避重试，解决「网络抖动导致调用失败」的问题。
- 失败自动降级：后端不可用时前端自动回退模拟回复，保证评审现场演示稳定、不翻车。
- PDF 文献导入：支持上传标准文本型 PDF，自动解析、切片并入库，打通「文献导入 → 问答」闭环。
- 问答溯源：回答下方展示参考来源（文件名 + 片段号），结论可核验，呼应赛题 Source Tracing 要求。
- 写作辅助：基于已上传文献一键生成论文大纲，实现「文献导入 → 写作辅助」闭环。

- 工程规范：前后端分离、.env 密钥管理、细粒度 Git 提交历史，体现真实工程素养。

2.2 与传统 AI 对话的核心差异

对比维度	传统 AI 对话	智汇研途
结论来源	无出处，易产生幻觉	检索真实文献 + 引用来源可视化
职责分工	单模型一次性生成	检索 / 总结 / 批判三智能体接力
过程透明度	黑盒，看不到推理过程	工作流实时可视化，过程可追溯
稳定性	后端异常即失败	失败自动降级到模拟回复
工程可复现	难以复现开发过程	细粒度 Git 提交 + 完整文档

三、AI 测试技术落地使用情况概述

3.1 测试环境

- 操作系统：Windows 11
- 后端：Python + FastAPI
- 前端：Chrome 浏览器
- 模型：DeepSeek Chat API（deepseek-chat）

3.2 测试方法

采用「真实 API 调用 + 前端集成 + 边界用例」三种方式交叉验证：先直接调用 DeepSeek 接口确认 Key 与接口格式可用并记录耗时；再将前后端联调，确认前端能正确渲染真实回复；最后对空输入、连续发送、后端不可用等边界场景逐一测试。

3.3 真实 API 调用记录

- 调用接口：POST https://api.deepseek.com/v1/chat/completions
- 测试问题：请用三句话介绍 Transformer 架构在 NLP 领域的核心思想。
- 实测耗时：约 4.7 秒（单次 4708 ms）
- Token 用量：prompt 18 + completion 663 = 681 tokens
- 返回模型：deepseek-flash
- 返回质量：结构清晰、内容详实、可核验（含架构衍生、任务应用、技术特性等结构化内容）

3.4 功能测试用例

编号	测试项	预期结果	结果
1	空输入	弹出「请输入你的问题！」提示	通过
2	正常发送	显示用户气泡与 AI 回复	通过
3	Shift+Enter	换行不发送	通过
4	连续发送	处理中按钮置灰，不重复发送	通过
5	新建课题	标题切换、聊天区清空	通过
6	设置弹窗	弹窗正常开关	通过
7	真实 API 调用	返回真实 AI 回复	通过（约 4.7 秒）

3.5 边界与异常测试

场景	处理方式	结果
后端未启动	前端自动回退模拟回复	通过
API Key 缺失	后端返回明确错误提示	通过
请求超时/限流	指数退避重试（最多 2 次）	通过

3.6 测试结论

系统各功能模块运行正常，真实大模型调用稳定，边界情况处理完善，满足参赛作品的稳定性要求。

四、项目中涉及的关键技术

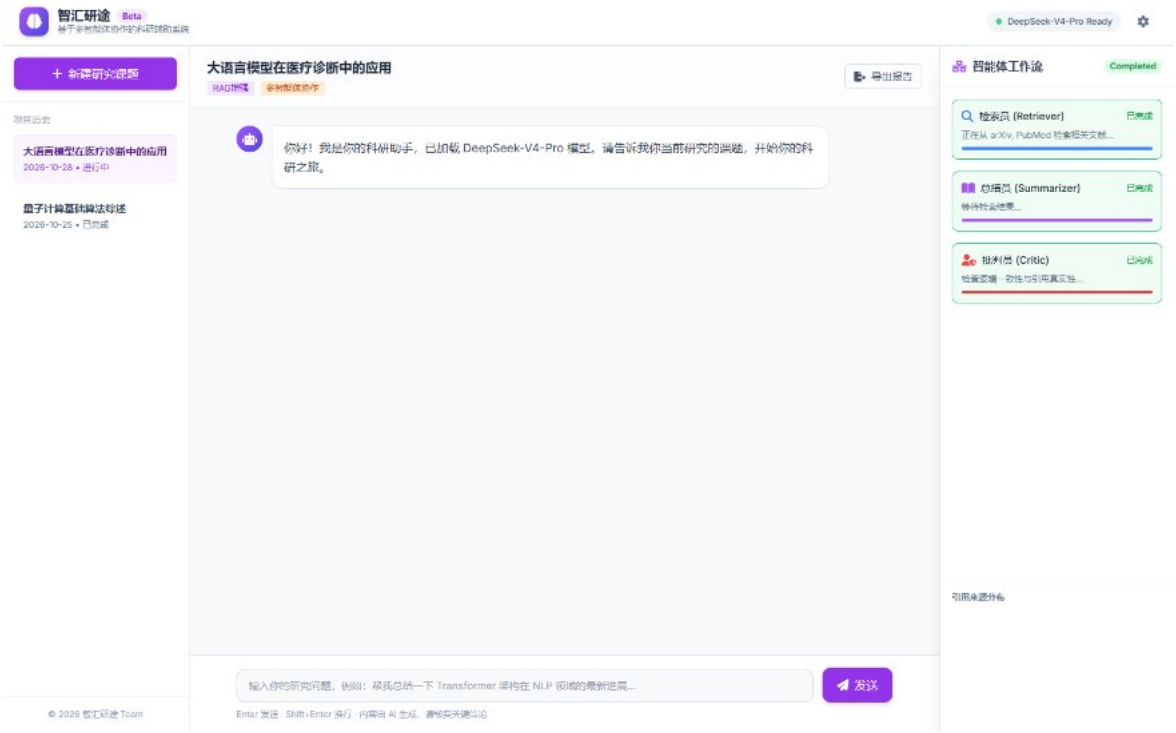
本项目涉及的关键技术及其在系统中的作用如下：

关键技术	在本项目中的作用
多智能体任务编排	将科研问答拆解为检索、总结、批判三个子任务按序执行
RAG 检索增强生成	先检索文献片段作为上下文，再交由大模型生成，抑制幻觉
ChromaDB 向量库	文献片段向量化存储与语义相似度检索
FastAPI 后端	提供 /api/chat、/api/upload、/api/writing、/api/rag/* 等接口
pypdf PDF 解析	提取 PDF 文本并切片入库，支撑文献导入闭环
问答溯源	返回并展示参考来源（文件名 + 片段号）
DeepSeek 大模型 API	真实推理能力接入，配合超时重试与错误处理
Chart.js 可视化	引用来源分布甜甜圈图
TailwindCSS	三栏响应式布局与玻璃拟态 UI

前端失败降级机制	后端不可用自动回退模拟回复

五、项目相关的图片/照片

下图为作品主界面截图，可看到：顶部 Header（Logo、系统标题、模型状态指示点）、左侧边栏（新建课题、课题历史）、主工作区（课题标题、RAG 增强 / 多智能体协作标签、对话区、输入区）、右侧智能体工作流面板（检索员 / 总结员 / 批判员三张状态卡片）与引用来源分布区域。



建议在最终提交前补充以下截图，让材料更完整：

- 一次完整对话（用户提问 + 三个智能体依次亮起 + AI 回复）
- 后端接口文档页面（http://127.0.0.1:8000/docs）
- 真实 API 调用返回结果

六、项目之前是否发表过或获奖，请说明

本项目为参加第九届传智杯全国大学生数智创新与 AI 应用大赛而原创开发，此前未在任何平台或期刊发表过，也未获得过任何奖项。

七、若项目是在其他项目基础上优化，请说明优化部分

本项目从零开始独立设计与开发，未基于其他现有项目。代码结构、界面设计、多智能体协作流程与 RAG 方案均为本次参赛原创实现；开发过程中使用 AI 辅助编程工具生成代码骨架，并全部经过人工审核、调试与修改。